Milestone 1 Package Sortation Range Build Report

Name: Kevin Berry
Section: IS-4543-002 / IS-4543-ON1
Date: October 14, 2026

Milestone Report

For Milestone 1, I built a virtual industrial control system based on a package sorter. My goal was to create a working environment that I could use for later cybersecurity testing. Before I could study attacks, I needed the sorter to work consistently and the network connections to follow a documented design. Otherwise, I would have difficulty telling whether a problem came from an attack or from a mistake in the simulation. I organized this report around the instructor's required proposal, activities, learning, and documentation sections.

My original milestone included three separate network segments, a firewall between them, an OpenPLC controller, conveyor and divert logic, two Python drive emulators, register documentation, proof of network isolation, a network diagram, and a video of a clean sort. I expanded the build to include three induction lanes, six drive emulators, three scanner tunnels, nine trailer destinations, and an operator interface. This report explains what I built, the problems I worked through, and the recorded results that support the build.

Key terms

Because this project comes from operational technology, I want to define the terms I use throughout this report. Operational technology, or OT, is the hardware and software that monitors and controls physical equipment, as opposed to IT systems that mainly handle data. An industrial control system, or ICS, is the set of OT devices that runs a process, in my case a package sorter. In a sorter, packages enter on induction belts, pass through a scanner tunnel that reads their barcodes, and are pushed off the main belt at a divert onto an outbound belt that feeds a trailer door. A package that cannot be routed in time recirculates, meaning it stays on the loop and comes back around instead of going to the wrong trailer. When a downstream belt is full, upstream packages wait; this is called backpressure. The programmable logic controller, or PLC, is the computer that runs the sorter's control logic, and variable frequency drives, or VFDs, control the motor speed of each belt. Photoeyes are light-beam sensors that detect when a package passes a point. SCADA, or supervisory control and data acquisition, is the layer above the PLC that collects process data and lets operators monitor it through a human machine interface, or HMI. Modbus/TCP and OPC UA are the two industrial protocols in my range: Modbus is an older, simple protocol for reading and writing numbered values with no built-in authentication, and OPC UA is a newer protocol that organizes data into named objects and supports security features. The Purdue model is a reference architecture that separates industrial networks into levels, from physical devices at the bottom up to business systems at the top. A zone is a group of devices with similar security needs, and a conduit is the controlled path between zones, which in my range is the firewall. XLe and ASX are the names I gave to my routing service and sort-plan service. A heartbeat is a value that changes on a regular schedule so other components can tell a service is still running.

Project Proposal

The following is my original project proposal. I have kept it here for reference, including the original milestone descriptions and AI use statement.

IS-4543 Cyber Attack & Defend
Course Project Proposal
Kevin

Idea 1: Segmented Sortation Attack and Defense Range

I will build a three-zone industrial control network modeled on package sortation and segmented along Purdue levels. I work in the UPS warehouse and this would be inspired from the setup there. An OpenPLC controller will run a conveyor and divert process, poll two emulated variable frequency drives over Modbus/TCP, and publish tags to an OPC UA server at the supervisory level. I will attack the process from the business zone, targeting the lack of authentication in both protocols. Finally I will instrument the range with Zeek and Suricata, add zone and conduit controls plus a validating protocol proxy, and measure which attacks each control detects versus blocks.

Milestone 1: Build the range. Create three isolated segments with a firewall VM as the conduit, install OpenPLC, write structured text for the conveyor and divert logic, and build two Python VFD emulators. Deliverables: network diagram, register maps, isolation proof, video of the line sorting cleanly.

Milestone 2: Supervisory layer and baseline. Stand up an OPC UA server bridging Modbus to Level 2. Capture several hours of normal operation and document the function codes, source addresses, and polling intervals defining baseline behavior. Deliverables: server code, pcaps, Zeek modbus.log, written baseline analysis.

Milestone 3: Attack. Run conveyor speed setpoint manipulation, package gap collapse, direct drive commands, function code enumeration, and unauthenticated OPC UA browse and write. Deliverables: commented attack code, one capture per attack, video of the missort counter climbing, measured impact table.

Milestone 4: Detect. Author Suricata rules and Zeek scripts covering unauthorized writes, physically impossible setpoints, exception response bursts, and connection rate anomalies. Test every rule against the milestone 3 captures. Deliverables: rule files, alert output, detection matrix including blind spots found.

Milestone 5: Harden and measure. Add nftables conduit rules, OPC UA sign and encrypt mode, and a validating Modbus proxy. Replay every attack against each configuration. Deliverables: control configurations, proxy source, results table separating detected from blocked, false positive counts, final report.

AI use: I used Claude to help design the range architecture, draft initial code for the PLC logic and emulators, and structure the milestones. The build, testing, and analysis are my own work, and AI-assisted code will be cited in each milestone submission

How the build changed

As I built the range, I learned that I wanted to model more of the sorting process. I expanded the original two-drive design to six drives, three induction lanes, three scanner tunnels, and nine trailer destinations. I

added a separate plant process, sensor timing and order checks, external routing services, and recovery checks. These additions gave me more realistic operating conditions to test, including delayed routing, stopped drives, missing sensor events, and service failures.

I also brought some supervisory work forward from Milestone 2 by adding the OPC UA bridge and HMI during the build. I needed that view to understand and verify what the packages and controller were doing. This does not mean I completed the later baseline, attack, detection, or hardening milestones. The original proposal remains my starting plan, while the sections below describe what I actually built and learned during Milestone 1.

The original proposal describes a lack of authentication in both protocols. More precisely, my initial lab uses plain Modbus/TCP and an OPC UA configuration that I have not yet hardened. OPC UA supports authentication, signing, and encryption [4]. The planned hardening work will apply those capabilities to my range.

Revised Plan for Milestones 2 through 5

Since I completed the OPC UA bridge and HMI during Milestone 1, I revised the remaining milestones to focus on measuring normal behavior, testing attacks, creating detections, and evaluating security controls. These are my proposed updates to the original plan.

Milestone 2: Establish normal operation and collect a baseline.
I will document normal traffic between the PLC, drives, scanners, and supervisory services. I will capture successful sorting, recirculation, and expected recovery behavior so I can distinguish normal events from attacks. Deliverables: packet captures, a communication map, operating results, and a written baseline analysis.

Milestone 3: Test attacks and measure their effects.
I will test unauthorized speed changes, direct drive commands, and OPC UA writes in my isolated lab. I will measure changes in package movement, spacing, loading, and alarms. Deliverables: commented attack scripts, packet captures, demonstration videos, and a table comparing each attack with normal operation.

Milestone 4: Create and test detections.
I will use Zeek and Suricata to identify unexpected sources, unauthorized writes, unusual command values, and repeated connection attempts. I will test the detections against attack captures and normal traffic. Deliverables: detection rules and scripts, alert examples, and a results table showing successful detections, missed attacks, and false positives.

Milestone 5: Apply controls and compare results.
I will restrict firewall access, enable OPC UA authentication and encryption, and add a Modbus proxy that checks permitted commands and values. I will repeat the attacks and normal-operation tests. Deliverables: control configurations, proxy code, and a final comparison of attacks detected, attacks blocked, and effects on normal sorting.

Summary of Activities

Building the network

I divided the lab into three separate virtual networks. I used the Purdue model as a guide for separating basic control from supervisory functions. NIST SP 800-82 Rev. 3 describes using levels or zones and boundary devices to control traffic between OT network segments [1]. I simplified that model into three lab zones: Level 1 contains the controller and simulated devices, Level 2 contains the supervisory services and operator interface, and Level 3 contains the analyst VM. Placing the analyst in Level 3 is a lab design choice for security testing. A VM is a virtual machine, which lets me run several separate computers on one physical workstation.

I used separate virtual networks routed through a firewall VM. I did not configure VLAN tags on a virtual switch. The firewall has one network interface in each zone and uses nftables to control traffic between the zones. I set its forwarding policy to drop traffic unless an allow rule applies. Device traffic within Level 1 stays on that network and does not pass through the firewall.

The Level 1 network uses 10.10.1.0/24. My PLC VM is 10.10.1.10, the six drive emulators use addresses 10.10.1.21 through 10.10.1.26, and the scanners use 10.10.1.27 through 10.10.1.29. The Level 2 SCADA VM is 10.10.2.10. The Level 3 analyst VM is 10.10.3.10. Each zone uses the firewall address ending in .1 as its gateway. XLe, ASX, the OPC UA server, and the HMI all run on the Level 2 SCADA VM at 10.10.2.10. XLe contacts ASX through the same VM's loopback address, 127.0.0.1, on TCP 8089. The OPC UA server listens on TCP 4840, and the HMI serves its operator page on TCP 8000. The plant process runs on the Level 1 drives VM, which owns the .21 through .29 aliases. It has no separate VM or IP address and sends its events to the PLC at 10.10.1.10 on TCP 502.

The initial firewall configuration, called Case A, intentionally allows broad access from the analyst VM to the control and supervisory zones. This gives me a starting point for the later attack milestone. I therefore describe Case A as a segmented baseline with intentional exposure. The stricter firewall policy belongs to the later hardening work.

Creating the controller and devices

I used OpenPLC, an open-source PLC runtime, for the controller [2] and wrote its control logic in Structured Text. A programmable logic controller, or PLC, repeatedly reads inputs, runs its logic, and updates outputs. In my range, the PLC tracks packages, triggers scanners, checks routing commands, handles faults, and updates trailer counters after a confirmed delivery.

I created six Python variable frequency drive emulators. A variable frequency drive, or VFD, controls the speed of a motor and reports its condition. Three of my emulators represent induction drives, and three represent outbound drives. They expose speed commands, speed feedback, and other operating values through Modbus/TCP. Modbus lets the services read and write numbered data locations called registers and coils. The Modbus messaging guide identifies TCP 502 as the standard server port [3].

I also added three scanner emulators that return synthetic barcode information when the PLC triggers them. These scanners do not read real camera images. They provide repeatable package identities so I can test routing and recovery. I documented the register maps so I could identify which values represented commands, feedback, package state, and faults.

Separating package movement from control logic

In the early version, the PLC advanced packages through its own simulated belt cells. That helped me test the basic logic, but it meant the same program controlled movement and decided whether the package had arrived. I added a separate Python plant process to make the process more useful for testing.

The plant process owns simulated package positions and raw sensor behavior. It moves packages using drive speed feedback and sends events to the PLC. The PLC checks those events before updating its accepted process state. This separation lets me test situations where a drive stops, a sensor event is missing, or the plant process becomes unavailable.

I added photoeyes to represent sensors along the package path. A photoeye detects when a package blocks a light beam. My simulation includes induction, tunnel, divert, outbound, and confirmation positions. The PLC checks sensor order and timing. I also added spacing and downstream capacity checks so packages can wait when the next section is occupied. This waiting behavior represents normal backpressure.

Adding routing and the operator interface

I used two services to provide external routing decisions: XLe, the routing service, and ASX, the sort-plan service. XLe reads the package information from the PLC and asks ASX for a destination. ASX uses a sort plan to return the trailer assignment. XLe checks the reply against the current package identity before sending a routing command to the PLC.

A barcode alone is not enough to identify a package. Two different packages can have the same barcode, and old replies can remain after a restart. I used run and package identity values, including an epoch, nonce, token, serial number, and scanner sequence, to keep commands tied to the correct package. These checks help reject stale or mismatched replies; they do not provide network authentication.

I added an OPC UA server to publish the process information in a form that the operator interface can use. OPC UA supports structured information models and a browsable address space [4]. I used that structure to publish named process objects instead of requiring the interface to interpret individual Modbus registers. My human machine interface, or HMI, displays package positions, drive information, alarms, and waiting conditions. It receives the process state through the supervisory layer, which publishes PLC-validated package information.

Following one package through the sorter

A package first enters an induction lane and receives one of the PLC's three available package slots. The plant process moves it toward the scanner tunnel. When the tunnel sensor changes state, the PLC triggers a scan. The scanner returns the package information, and XLe requests a destination from ASX.

After XLe checks the reply, the PLC accepts the route and the package continues toward the divert and outbound section. The plant reports the later sensor events. The PLC increases the trailer counter only after it receives the expected confirmation for that package. Choosing a destination does not count as a completed delivery. If a routing decision arrives too late, the package recirculates instead of being sent to an unconfirmed destination.

Description of Learning Completed

Resets must include the connected devices

One problem appeared when I reset the PLC but left the scanners running. The scanners still held information from the previous run, so a new request could receive an old response. I learned that resetting one component does not automatically reset the whole process.

I added a prepare and reset handshake between the PLC and scanners. The prepare step replaces old responses, and the reset step clears scanner history and returns an acknowledgement for the current run. The PLC waits for all three scanners before allowing operation. If a scanner does not respond within the allowed time, the sorter stops and identifies the failed tunnel. The operator must acknowledge the fault and retry after service is restored.

Restarts need reliable package records

I found that XLe could lose track of accepted packages when it restarted because its original records were kept in memory. I changed it to rebuild active package information from the PLC and record completed outcomes in a SQLite database. This lets it recover without treating an accepted package as a new routing request.

I also found that a cold PLC restart could reuse identity values from an earlier run. I added a durable run epoch so the database could distinguish those runs. This taught me to consider what survives a process restart and what returns to its initial value. A system can appear correct during one run and still fail when several components restart at different times.

A heartbeat shows whether a service is still responding

I added heartbeat checks for services that the process depends on. A heartbeat is a value that changes regularly so another component can tell whether the service is still responding. If the expected updates stop, the PLC treats the service as unavailable.

In the recorded plant failure test, stopping the plant process caused a heartbeat fault. The PLC stopped induction, the HMI held the last validated package positions, and trailer counters stayed at zero. Restarting the plant did not safely reconstruct packages that had already been moving. Recovery required a stopped reset. I learned that restarting a service is only part of recovery; the process also needs a known state before operation resumes.

I need to prove the cause of a network failure

Network testing taught me to separate a blocked connection from a service that is not listening. A connection refusal does not prove that the firewall blocked traffic. I checked the expected connection result together with firewall counters to support the conclusion. For the documented blocked probe, the connection timed out and the default drop counter increased.

I also learned to separate functional results from cleanup results. Some package scenarios worked, but the overall validation failed because the lab did not return to the required baseline. I kept those failures in the records instead of calling the whole run successful. I improved the restoration checks to distinguish values that must reset from values that can keep changing. For example, QW221 is the next serial number, so

requiring it to return to zero caused a false failure. I corrected that check and used an operator reset with the scanner handshake to clear actual leftover process state. After recovery, the lab passed the documented 215-field baseline check. This gave me a clear way to verify that a test had finished and the lab was ready for the next run.

Sample code and why I am proud of it

One part of the project I am proud of is the check that XLe performs before it accepts a routing reply. The following function comes from services/xle.py at commit 1e271ed [5]. It checks whether the reply belongs to the request and whether the destination is valid before the routing service uses it.

```
def validate_decision(request, response):
    if response.get("request_id") != request["request_id"] or \
       response.get("package_id") != request["package_id"]:
        return None, "stale_or_mismatched_response"
    if response.get("decision") == "no_decision":
        return None, "unknown_package"
    destination = response.get("destination")
    if response.get("decision") != "route" or type(destination) is not int or \
       not 1 <= destination <= 9:
        return None, "invalid_decision"
    return destination, None
```

Listing 1. Routing reply validation in services/xle.py

The first check compares both the request ID and package ID. If either value does not match, the function returns a reason instead of accepting the reply. This matters because a delayed reply may belong to an earlier request. The next check handles a package for which the sort-plan service has no destination. The final check requires a route decision and a whole-number destination from 1 through 9. Only a reply that passes every check returns a destination.

I am proud of this part because it reflects how my thinking changed during the build. At first, getting a package to move and reach a trailer was the main goal. As the range grew, I had to consider old replies, repeated barcodes, and invalid values. This function puts those checks in one clear place and gives me a specific reason when a reply is rejected. It is only one layer of the design: the PLC still checks the full package identity and confirms the delivery. Together, those checks make the range more useful for later attack testing.

Using the lab in interviews

This lab has helped me get multiple interviews, and I have already had to explain its current state during them. I have discussed how I separated the networks, how the PLC and HMI work together, and how I investigated failures. Those conversations gave me a reason to understand the choices behind the project and explain them in plain language.

I am proud that something inspired by my work in the UPS warehouse has become a project I can discuss with employers. It gives me concrete examples of networking, troubleshooting, and system recovery. Speaking about the lab has also helped me be precise about what I implemented, what I tested, and what remains planned.

Documentation of Work Completed

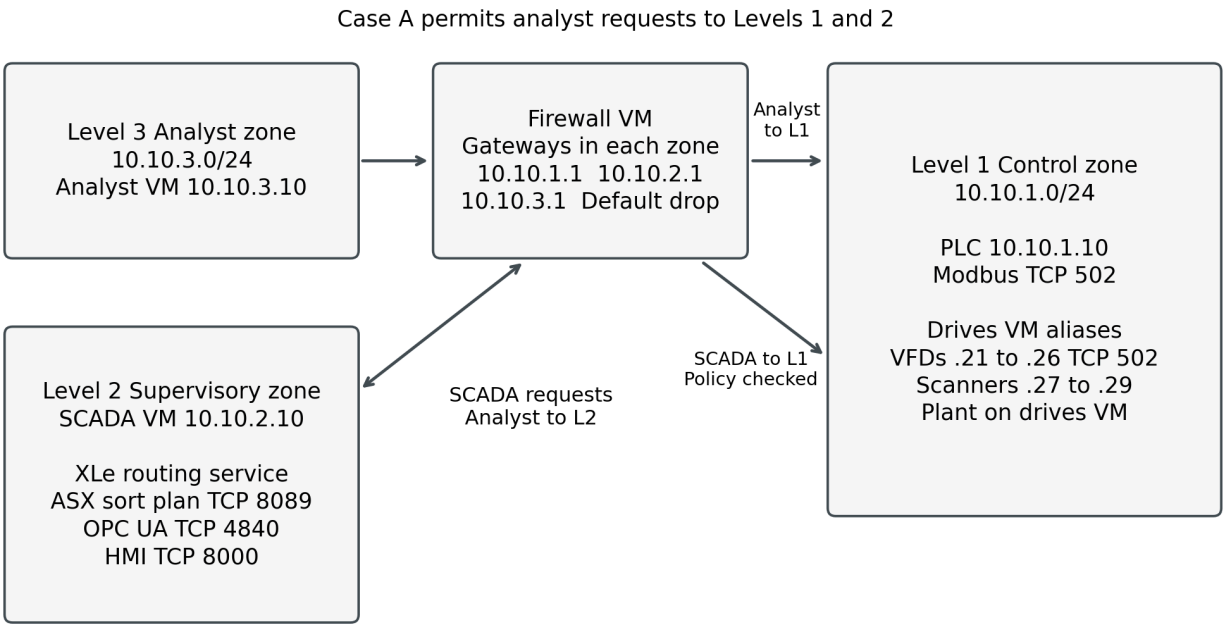


Figure 1. Network layout of my package sortation range and main Case A conduits

Recorded process tests

I documented the build in the repository and recorded tests for normal sorting, routing failures, device behavior, and recovery. The following results show how I checked the main functions. My project code and supporting evidence are available in the public repository at commit 1e271ed [5].

Test	Recorded result	What I verified
Normal three lane sort	Packages from lanes 1, 2, and 3 reached trailers 2, 5, and 8.	The scanner, route services, PLC, and delivery confirmation worked together.
Package capacity	Three packages occupied slots at once; a fourth waited for a release.	The PLC enforced its three package limit.
Late routing reply	The package recirculated without a wrong trailer load.	The routing timeout handled the missing decision safely.
Missing confirmation	The trailer counter remained zero.	An assigned destination was not treated as a completed delivery.
Stopped induction drive	The package held at zero speed feedback and moved after feedback returned.	Movement followed the simulated drive feedback.
Missing tunnel sensor	The PLC latched a sensor order fault and trailer counters stayed zero.	The controller checked the expected sensor sequence.

Network and code validation

My recorded network test checked 36 source, destination, and port combinations. All 36 matched the declared Case A policy: 30 were allowed and 6 were denied. I also recorded a successful SCADA-to-PLC connection on TCP 502 with an increasing allow counter, and a blocked SCADA-to-scanner probe with an increasing drop counter. These results support the documented firewall rules for the tested paths.

I used host tests to check the controller logic and Python services, then live tests to check communication between the VMs. My validation records show 183 passing Python tests, a passing PLC test harness, passing syntax checks, and a passing live deployment check covering 33 components. The host tests helped find logic problems, while the live tests checked whether the deployed services worked together over the lab network.

Repository evidence

Repository and code: https://github.com/berryk6490/ics-sorter-range/tree/1e271ed

I kept the network design and connection results in NETWORK_DIAGRAM.md and NETWORK_BASELINE.md. MILESTONE1_READINESS.md summarizes milestone readiness. The register maps document the interfaces between the controller, drives, scanners, plant, and supervisory services. These records provide the details needed to repeat the tests and interpret the values shown in the operator interface.

I documented reset and recovery behavior in RUN_IDENTITY.md, XLE_RECOVERY.md, and XLE_LIVENESS.md. I recorded package movement and sensor checks in PLANT_SLICE.md and STATEFUL_PHOTOEYES.md, and waiting behavior in ACCUMULATION_BACKPRESSURE.md. Machine-readable results are stored under tests/evidence, with rendered test images under tests/artifacts. These records are included in the linked project repository [5].

Completion and limitations

I completed the technical build described in my original milestone and added more device and recovery behavior than I initially planned. The range includes three separate network zones, a firewall conduit, the OpenPLC sorter, six drive emulators, three scanners, an HMI, register documentation, and recorded network tests.

My range is a simulation, so it has limits. Package motion is simplified and does not calculate friction, belt slip, or collision forces. The scanners generate test data, and the sensors do not model real electrical noise. I also do not emulate a separate safety PLC or physical emergency-stop circuit. These limits define what I can claim from the tests.

Milestone 1 gave me a working process baseline for the later security work. I now have normal routes, expected network connections, and documented failure behavior to compare against controlled attacks. My next milestones can use that baseline to study unauthorized commands, create detections, and evaluate hardening changes.

AI use during this milestone

As stated in my proposal, I used Claude for help with the architecture, initial PLC and emulator code, and milestone planning. I used ChatGPT to help organize, draft, and revise this report from my project records. The AI-assisted starting code was the conveyor and divert PLC logic that developed into Sorter.st (with the earlier version kept as sorter.st) and the VFD emulator in devices/vfd.py.

References

[1] Stouffer, K., et al. (2023). Guide to Operational Technology (OT) Security. NIST SP 800-82 Rev. 3. Section 5.2.3.1, Network Segmentation and Isolation, and Figure 16.

https://doi.org/10.6028/NIST.SP.800-82r3

[2] OpenPLC Project. OpenPLC Runtime version 3. Project README and source repository.

https://github.com/thiagoralves/OpenPLC_v3

[3] Modbus Organization. MODBUS Messaging on TCP/IP Implementation Guide V1.0b.

https://www.modbus.org/file/secure/messagingimplementationguide.pdf

[4] OPC Foundation. Unified Architecture. Information modeling, access, and security overview.

https://opcfoundation.org/about/opc-technologies/opc-ua/

[5] Berry, K. Segmented Sortation Attack and Defense Range. Project code and evidence at commit 1e271ed.

https://github.com/berryk6490/ics-sorter-range